Data-Driven Autonomous Meal Session Detection Using Smartwatches

1st W. Camilo Ariza-Zambrano
AI R&D Team
Samsung R&D Institute Brazil
Campinas, Brazil
ORCID:0000-0001-5151-8406

2nd Vinicius H. Cene
AI R&D Team
Samsung R&D Institute Brazil
Campinas, Brazil
ORCID:0000-0002-5647-1296

3rd Dayane Oliveira Gonçalves
AI R&D Team
Samsung R&D Institute Brazil
Campinas, Brazil
ORCID:0000-0002-1943-9561

4th Carlos Caetano
AI R&D Team
Samsung R&D Institute Brazil
Campinas, Brazil
ORCID:0000-0002-1546-3740

5th Otávio A. B. Penatti
AI R&D Team
Samsung R&D Institute Brazil
Campinas, Brazil
ORCID:0000-0002-0171-4430

2024 IEEE 20th International Conference on Body Sensor Networks (BSN) | 979-8-3315-3014-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/BSN63547.2024.10780654

***Abstract*—Poor dietary habits are frequently associated with diverse health conditions. Although there are apps designed to monitor eating habits, they require manual annotation, usually leading to user disengagement. This paper presents a data-driven solution using smartwatch sensors to automate meal session detection, addressing limitations of wearables such as a laterality unified model, capturing meal sessions with different durations, model compression to enable embedding, and dynamic inference times to reduce energy consumption. We tested two pipeline variations on a dataset of 91 subjects in a free-living scenario for 12 days, resulting in 93% Precision and 82% Recall, indicating the feasibility of free-living meal session detection. We also developed an application running on Samsung smartwatches to assess the solution feasibility with limited resources. This solution can be used as the building block to track eating-related phenomena such as autonomous dietary guidance and real-time non-invasive glucose monitoring, which remain unsolved problems in literature.**

***Index Terms*—Meal detection, Human activity recognition, Wearable devices, Inertial signal processing.**

I. Introduction

The accurate detection and journaling of meal sessions are crucial for health applications such as personalized nutrition and monitoring of food-related chronic diseases (*e.g.*, obesity and metabolic disorders) [1]. Moreover, avoiding poor eating habits can mitigate conditions like hypertension and other cardiovascular diseases. The current smartphone-based applications often rely on constant user annotations to promote the longitudinal tracking of eating habits. Due to the continuous interaction requirement, users tend to disengage, precluding proper monitoring of meal session events. Despite the growing interest of the industry and the scientific community in this topic in recent years, and although some literature advances, there has yet to be a consensus on how to automate this event annotation to derive this important health indicator.

Since eating patterns correlate with upper-limb movements, most solutions use inertial data from sensors attached to users' arms/wrists to map the meal sessions [2]. Studies using Inertial Measurement Unit (IMU) data to detect eating activities explore two main alternatives [3], [4]. The first approach relies on mapping the micro-movement sequences involved (e.g., fork to plate, cutting, biting, among others) in the eating pattern and input this sequence into temporal models - Long Short-term Memory (LSTM) networks or similar structures - to check the temporal coherence, increasing or decreasing the classifier probability of estimating a meal-intake cycle event [3]. Another approach is processing the occurrences by a post hoc heuristic-based technique (e.g., the number of bites in a given time interval) to classify a specific period as an eating session [3], [5]. Data-driven approaches usually rely on minimal signal pre-processing and end-to-end automated deep-learning pipelines where the raw or normalized signals enter a temporal model, generating an output related to the eating probability [4]. Alternatives based on preliminary eating detection and further micromovement classification before a final classification are also possible and explored in the literature [2].

A robust automated meal session detection solution capable of addressing free-living conditions while avoiding false detections is still to be developed. This paper tackles this complicated and unconstrained scenario by offering practical alternatives and a Proof-of-Concept (PoC) implementation in an off-the-shelf device. The solution developed is detailed in the following sections.

II. Experimental Methodology

This section presents the data gathering protocol and complementary data used in training and validating our system and the different architectures explored. Our proposed meal detection pipeline uses, in part, the model proposed by [2] and a Recurrent Neural Network for performance comparison against the literature. It is composed of three main stages: pre-processing, modeling, and post-processing. Figure 1 illus-

Authorized licensed use limited to: San Francisco State Univ. Downloaded on September 02,2025 at 20:33:10 UTC from IEEE Xplore. Restrictions apply.

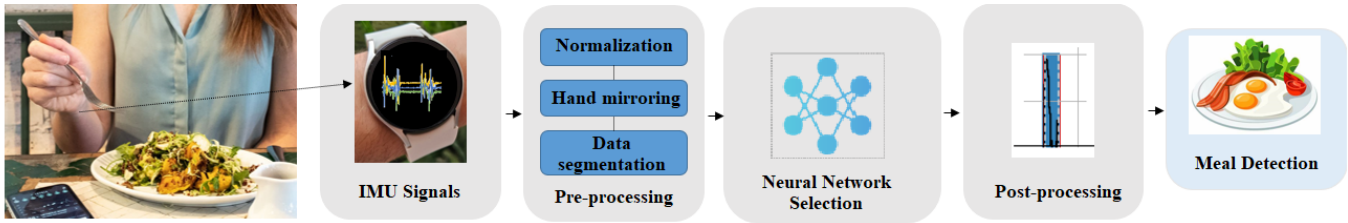

Fig. 1. Overview of the autonomous meal session detection solution based on IMU signals, neural networks, and post-processing stages.

trates the solution pipeline and its different blocks, from data recording to the event output.

A. Dataset

We used various literature datasets and a proprietary data collection to train and validate the solution. Our proprietary dataset comprises data from 91 subjects in all-day free-living conditions for 12 days while performing breakfast, lunch, and dinner. The available time of sensor data is roughly 13,000 hours with nearly 200 hours of eating activity while the remainder describes several unconstrained activities. Our proprietary dataset gathers data from IMU sensors from a Samsung Active 2 smartwatch collected at 20 Hz from both users' wrists. In our dataset, 78 subjects are right-handed, three are left-handed, and 10 have undisclosed laterality.

In addition to our dataset, three publicly available literature datasets (FIC [3], FreeFIC [4], and IJS [5]) were used to augment the data and enhance model robustness during training. The three dataset added 36 subjects and more than 550 hours (eating and other activities) of sensor acquisition to our initial data. Enriching this data in training is important considering the unbalanced nature of a free-living dataset, where the eating activities are rare and several activities (flossing, brushing teeth, among others) hold similarity with meal sessions.

B. Meal detection pipeline

1) Pre-Processing Stage: The pre-processing stage is mainly related to basic processing and data preparation. It is divided into three main steps: (i) Z-score normalization, (ii) Dominant-hand mirroring, and (iii) Data segmentation. While normalization relies on adapting the IMU signal ranges from different datasets to the same reference, the mirroring and segmentation functions are heuristic-oriented. The dominant hand refers to the predominant hand used in a feeding session. Since the human motion activity in feeding tends to be laterality-dependent, the main idea of the dominant-hand mirroring function is to transpose the left-hand data (from left-handed users) into right-hand coordinates before training the model. Thus, this step is very important to make the model side-agnostic. Once the user indicates the dominant hand, the processing occurs autonomously. The data segmentation is the final step of signal processing, where signals are down-sampled and segmented using sliding windows. To determine the optimal down-sampling and segmentation parameters that maximize the accuracy rate, we used the Automated Machine Learning Neural Network Intelligence (NNI) framework [6]. The resulting parameters were 5Hz and sliding windows with 1-minute and 1-second increments.

2) Modeling: We evaluated different machine learning models in this step and here we report only the ones with higher accuracy rates, which are: (i) a Convolutional Neural Network (CNN)+LSTM combination that takes advantage of the structural organization of the sensors while encoding their temporal and spatial relations [2], and (ii) a Recurrent Neural Network (RMM)-based model that encodes the temporal dynamics of the movements composed of two fully-connected layers to learn feature representation, followed by two-layer LSTM layers to capture the temporal patterns, followed by two fully-connected layers to learn temporal relations generated from the LSTM and a final fully-connected layer for prediction. Figure 2 provides an overview of the architecture of both models. The models' outputs are binary classifications (eating vs. non-eating) based on probabilities calculated for each input frame. We performed model optimization using Bayesian Optimization and Hyperband (BOHB) [7] that reduced the number of parameters from 381k to 21.4k (5.6% of the original size) and from 907k to 25.5k (2.8% of the original size) for the CNN+LSTM and RNN models, respectively.

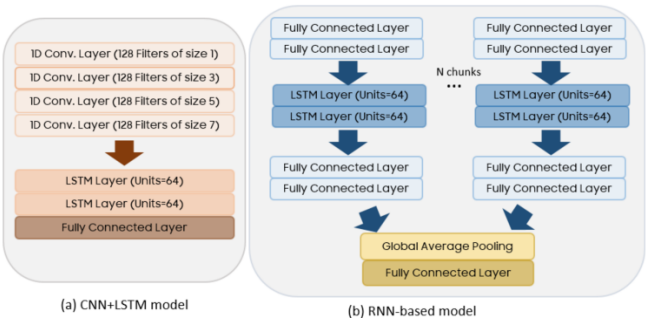

Fig. 2. Machine learning architectures for modeling the structural and temporal dynamics of the input signal.

3) Post-Processing Stage: This stage aims to improve the class transitions, enhancing the predictions to determine more precisely when a meal-intake session started and ended. This problem is equivalent to time series segmentation, whose values have a similar mean or standard deviation. We used the entire validation dataset and the NNI framework to define the thresholds most appropriate for eating event detection and deactivation.

Authorized licensed use limited to: San Francisco State Univ. Downloaded on September 02,2025 at 20:33:10 UTC from IEEE Xplore. Restrictions apply.

4) PoC application for Samsung's Galaxy Watch: We compressed the models using PyTorch Lite to enable the model porting to the application level of Samsung's Galaxy Watch 4 and 5 models. We implemented a PoC to show the feasibility of running the detection directly on the smartwatch while also logging data for further analysis. At the back-end of our PoC, which targets Android Software Development Kit (SDK) 12, Application Programming Interface (API) level 31, a background worker keeps track of the whole meal session detection algorithm, composed of a sensor listener, data buffering, logging, and inference thread. The smaller model (CNN+LSTM) is loaded using PyTorch Android API v1.10.0 and tested using accelerometer and gyroscope data collected at 5Hz in 300s windows. Thus, there is a necessary buffering of five minutes before the prediction starts. Figure 3 presents the PoC application pipeline diagram.

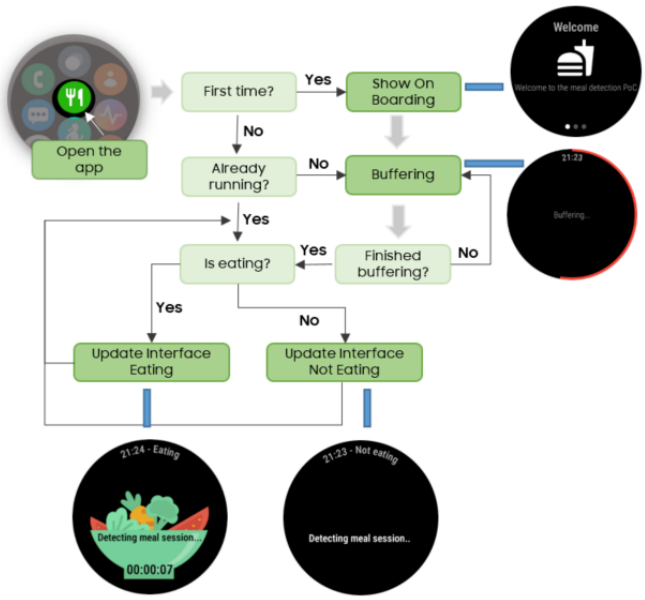

Fig. 3. Diagram of the PoC application: The model starts predictions after sensor access authorization and a five-minute buffer phase. Following the detection of meal session events, the screen displays an animation and timer indicating the duration of the current meal session. Predictions are also executed in a background process, ensuring continuous monitoring.

III. Results and Discussion

We employed two main tests to evaluate the meal detection pipeline. The first test divides the data into train, validation, and test splits (63, 14, and 14 subjects, respectively). The second test uses a Leave-One-Subject-Out (LOSO) cross-validation to test the method in a more realistic scenario. Both tests included augmentation separately in the training stage, with the derived results in Table I. The Precision of both methods is higher than the Recall, proving the method's sensitivity in detecting meal-intake events. The method sensitivity can be regulated using post-processing to adjust the trade-off between Precision and Recall. We adjusted this balance to avoid false detections, at the cost to losing some few real meal sessions. Regarding the Neural Network methods, RNN presented higher accuracy than the CNN+LSTM combination, and we attribute this to be a consequence of RNN's temporal capacity for tracking repetitive hand-to-mouth movements, which are very common in meal sessions.

TABLE I
Results for the different test protocols.

Protocol	Precision		Recall	
	CNN+LSTM	RNN	CNN+LSTM	RNN
Train/Val/Test	0.908	0.932	0.768	0.757
LOSO	0.917	0.930	0.815	0.823

For a comparison with existing state-of-the-art approaches, we selected the LOSO protocol that represents a more realistic use-case scenario. As baselines, we used the work of *Heydarian et al.* [2] and *Kyristsis et al.,* [3], that are based on a CNN+LSTM combination to predict meal session events, similar to the proposed method. *Kyristsis et al.* [3] also trained a CNN on sub-gesture annotations, whose information is not available in our data. The third baseline, proposed by *Stankoski et al.* [5], uses a variation of CNN-based structures to map bites and, consequentially, meal intake events. Although the results of each method are not directly comparable due to the use of different datasets for testing, we put them side-by-side in Table II in order to visualize together how they performed on their respective experiments.

TABLE II
Performance results of proposed methods and three state-of-the-art approaches.

Method	Dataset (# of subjects)	Precision	Recall	F1
Ours (CNN+LSTM)	91 (free-living)	0.92	0.82	0.86
Ours (RNN)	91 (free-living)	0.93	0.82	0.87
Heydarian et. al. [2]	100 (lab)	0.76	0.79	0.78
Kyristsis et. al. [3]	12 (lab)	0.89	0.93	0.91
Stankoski et. al. [5]	12 (free-living)	0.85	0.81	0.82

The results indicate higher performance of our approach than the state-of-the-art *Heydarian et al.* [2] and *Stankoski et al.* [5] approaches. The superior overall performance reported by the *Kyristsis et al.* [3] may derive from a more restricted and controlled experiment, without the free-living conditions where misclassification occurs more frequently. Thus, although *Kyristsis et al.* [3] excel in mapping sub-gestures to meal sessions it was not possible for us, determine how efficiently they would detect meal events in the wild.

Embedded solutions must be not only accurate but also power-efficient. Thus, as we performed validation tests of our PoC regarding meal session detection, we also tackled the power consumption. Since different smartwatch models have different battery capacities, we tested six devices of four models to analyze energy consumption. During our investigation, we defined the inference interval time as the controlled variable that will impact energy requirement the most since, at every activation, the whole deep-learning model runs the eating probability in near real-time conditions. The models

Authorized licensed use limited to: San Francisco State Univ. Downloaded on September 02,2025 at 20:33:10 UTC from IEEE Xplore. Restrictions apply.

tested were Galaxy Watch 4 (SM-R890), 2 Galaxy Watch 4 (SM-R870), Galaxy Watch 5 (SM-R910) and 2 Galaxy Watch 5 (SM-R900). The larger smartwatch models (SM-R890 and SM-R910) have roughly 30% more capacity than the SM-R870 and SM-R900 models (247 mAh x 361 mAh). The tests considered free-living with the device in an office environment on different days for each configuration, where the energy consumption was measured using the Battery Historian app [8] after we charged the devices to 100%. We calculated the normalized power consumption as the ratio between the specific app and the overall smartwatch consumption. Figure 4 presents the results derived from this test.

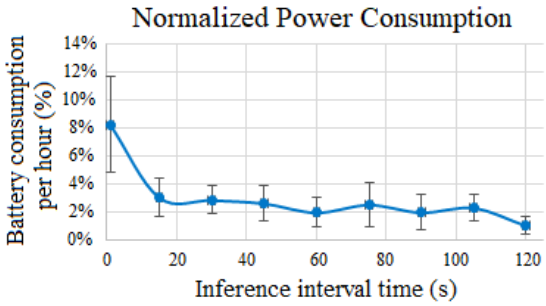

Fig. 4. Battery consumption x inference time: the blue line represents the battery consumption that is drastically reduced according the inference interval since running deep-learning models require considerable amount of the smartwatch resources. The standard deviation associated presents the difference across the different smartwatch models which is expected according their different battery capacities.

Although a 2-minute leads to reduced battery consumption (approximately 1.27% per hour), it also causes the method to be less responsive. Thus, we used a strategy of dynamical inference based on the eating probability threshold. We begin the app running with sparse 2-minute intervals, and once the probability of eating exceeds a specified value, we switch the inference to occur every second. The dynamical inference allowed us to save most of the battery consumption since, most of the time, users are not expected to eat. However, once a possible eating event occurs, the app can track it with enough resolution, as presented on Figure 5.

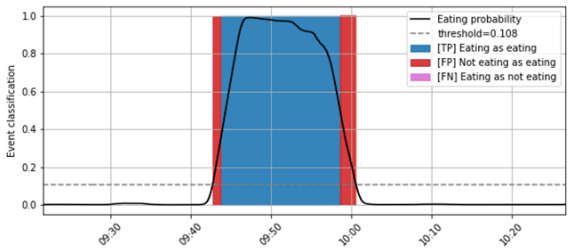

Fig. 5. Example of meal detection using a Samsung Galaxy Watch 4 and the dynamical inference approach. The blue section represents a true eating identified region while the red section refers to false negatives. The threshold is represented by the dashed constant line while the calculated probability is represented in the bold black line.

Figure 5 presents a meal detection result derived from this technique, where we maintained reasonable detection rates while strategically using power resources. Since the app test was a more exploratory initial test, we did not calculate the specific metrics from all acquired data and evaluated it qualitatively. We will expand this study by implementing the methods in lower hardware levels to increase energy efficiency and perform new data acquisition studies to assess the solution in this new context.

IV. Conclusion

This paper presented a data-driven pipeline to autonomously detect meal session events based on wearable sensor data and deep learning models. The pipeline includes elements to address the challenges of free-living meal session detections in wearable devices, like hand-mirroring approach, to enable a side-agnostic model, data-adjusted input buffers to machine learning models for different sizes of meal session durations, model compression with Bayesian optimization, to allow model embedding, and dynamic inference times to reduce energy consumption. We validated the pipeline with a dataset of 13,000 hours of sensor data and compared it with the literature's baselines, achieving competitive performance with a more efficient approach. Our smartwatch implementation demonstrated the feasibility of running the proposed pipeline on Samsung Galaxy Watch devices with low battery consumption. In future works, we plan to investigate the influence of other sensors and metrics, such as heart rate variation and skin temperature, to compensate for feeding events without consistent upper-limb movement representation and explore differences in eating and drinking patterns.

Acknowledgment

This work was funded by Samsung Eletrônica da Amazônia LTDA under the terms of Brazilian Informatics Law 8.248/91.

References

[1] S. Zhang, W. Stogin, and N. Alshurafa, "I sense overeating: Motifbased machine learning framework to detect overeating using wrist-worn sensing,"*Information Fusion*, vol. 41, pp. 37–47, 2018.

[2] H. Heydarian, P. V. Rouast, M. T. Adam, T. Burrows, C. E. Collins, and M. E. Rollo, "Deep learning for intake gesture detection from wrist-worn inertial sensors: The effects of data preprocessing, sensor modalities, and sensor positions,"*IEEE Access*, vol. 8, pp. 164936–164949, 2020.

[3] K. Kyritsis, C. Diou, and A. Delopoulos, "Modeling wrist micromovements to measure in-meal eating behavior from inertial sensor data,"*IEEE journal of biomedical and health informatics*, vol. 23, no. 6, pp. 2325–2334, 2019.

[4] K. Kyritsis, C. Diou, and B. Delopoulos, "A data driven end-to-end approach for in-the-wild monitoring of eating behavior using smartwatches," *IEEE Journal of Biomedical and Health Informatics*, vol. 25, no. 1, pp.22–34, 2020

[5] S. Stankoski, M. Jordan, H. Gjoreski, and M. Luštrek, "Smartwatch-based eating detection: Data selection for machine learning from imbalanced data with imperfect labels," *Sensors*, vol. 21, no. 5, p. 1902, 2021

[6] Microsoft Inc. (2024) Neural network intelligence: An open source automl toolkit for hyperparameter optimization, neural architecture search, model compression and feature engineering. [Online]. Available: https://nni.readthedocs.io/

[7] S. Falkner, A. Klein, and F. Hutter, "Bohb: Robust and efficient hyperparameter optimization at scale," in *International Conference on Machine Learning*. PMLR, 2018, pp. 1437–1446.

[8] Android Developer. (2019) Profile battery usage with batterystats and battery historian.

Authorized licensed use limited to: San Francisco State Univ. Downloaded on September 02,2025 at 20:33:10 UTC from IEEE Xplore. Restrictions apply.